

REACT HOOK

useState()

A Complete Beginner's Guide

Explained So Simply, Even a Child Can Understand!

Super Simple

With Code

Beginner Friendly

What's Inside This Guide?

- ✓ What is useState and why do we need it?
- ✓ How state works - with super simple analogies
- ✓ Real code examples step by step
- ✓ Common mistakes and how to fix them
- ✓ Fun mini challenges to test yourself

What is useState?

Think of it like a Scoreboard!

Imagine you're playing a video game.

The game shows your SCORE on screen. Every time you defeat an enemy, the score UPDATES!

useState is EXACTLY that score box for your React app.

It holds a value, and when that value changes, React automatically updates the screen!

Simple Definition

useState is a React Hook that lets you add a "memory" to your component.

It stores data that can CHANGE over time - like a counter, a name, or true/false.

* Memory Box

Gives your component a special memory box to store data

* Auto Update

When data changes, React automatically redraws the screen

* Value Storage

Store numbers, text, true/false - almost anything

■ Why Do We Need useState?

Without useState, React has NO way to notice changing data. Let's see the problem:

```
function Counter() {  
  let count = 0; // just a normal variable  
  function addOne() {  
    count = count + 1; // changes the variable...  
    // ...but React doesn't KNOW about it!  
  }  
  return <button onClick={addOne}>Count: {count}</button>;  
}
```

! PROBLEM: Normal variables don't tell React when they change. React is like a sleepy guard - it won't wake up unless you use special tools like useState.

The Magic Syntax

Step 1: Import useState from React



```
// Always import useState at the top of your file
import React, { useState } from 'react';
```

Step 2: Use it inside your component



```
const [value, setValue] = useState(initialValue);
//      |           |           |
//  current  function  starting
//   value   to update  value
```

value

Current value
(what React
remembers now).
e.g. count = 5

setValue

Updater function.
Call it to change
the value.
e.g. setCount(6)

initialValue

Starting value
when component
first loads.
e.g. 0, "", false

Real Examples - Different Data Types



```
// Number - for counters, scores, age
const [count, setCount] = useState(0);

// String - for text, names, messages
const [name, setName] = useState('');

// Boolean - for true/false, on/off
const [isLoggedIn, setIsLoggedIn] = useState(false);

// Array - for lists of items
const [todos, setTodos] = useState([]);
```



TIP: The naming convention is value and setValue. For example: count -> setCount, name -> setName. This pattern makes your code very readable!

Your First useState App

The Goal

Build a button that shows a number. Every time you click it, the number goes UP by 1.
This sounds simple, but it teaches you EVERYTHING about how useState works!

```
import React, { useState } from 'react';

function Counter() {
  // 1. Create a state variable called 'count', starts at 0
  const [count, setCount] = useState(0);

  // 2. Function that runs when button is clicked
  function handleClick() {
    setCount(count + 1); // add 1 to current count
  }

  // 3. Show the count on screen
  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={handleClick}>Click Me! +1</button>
    </div>
  );
}
```

What Happens Step by Step?

1

App Loads

count starts as 0. Screen shows
Count: 0

2

User Clicks

handleClick runs, setCount(0 + 1) is
called

3

React Wakes Up

React sees state changed and
re-renders

4

Screen Updates

count is now 1. Screen shows
Count: 1

! KEY INSIGHT: You NEVER change count directly (like `count = count + 1`). Always use `setCount()`. This is how you wake up React and tell it to update the screen!

More Fun Examples!

Example 2 - Toggle (On/Off Switch)

A light switch! Click the button to toggle a light ON or OFF.

```
function LightSwitch() {
  const [isOn, setIsOn] = useState(false); // starts OFF

  function toggle() {
    setIsOn(!isOn); // flip true to false, false to true
  }

  return (
    <p>Light is: {isOn ? 'ON' : 'OFF'}</p>
    <button onClick={toggle}>Toggle Light</button>
  )
}
```



!isOn means 'the opposite of isOn'. If isOn is true, !isOn is false, and vice versa. It flips the value!

Example 3 - Text Input (Live Typing)

Show what the user types, in real time! (Hello, {name}! updates as you type)

```
function NameInput() {
  const [name, setName] = useState(''); // starts empty

  function handleChange(event) {
    setName(event.target.value); // what the user typed
  }

  return (
    <input
      placeholder="Type your name..."
      onChange={handleChange} />
  )
}
```



event.target.value is the current text in the input box. Every keystroke runs handleChange, which calls setName(), and React instantly updates the <p> tag!

Common Mistakes

✗ Mistake 1: Mutating State Directly

Wrong

```
// Never do this!  
count = count + 1;  
items.push('new'); // mutates array!
```

Correct

```
// Always use the setter!  
setCount(count + 1);  
setItems([...items, 'new']);
```

Always use the setter function (setCount, setName, etc). Never change the value directly.

✗ Mistake 2: Using Old State Value

Wrong

```
// Might use a stale value!  
setCount(count + 1);  
setCount(count + 1); // same old count!
```

Correct

```
// Function form gets latest value  
setCount(prev => prev + 1);  
setCount(prev => prev + 1); // safe!
```

When new state depends on old state, use setState(prev => prev + 1) to always get the latest value.

✗ Mistake 3: useState Outside a Component

Wrong

```
// useState must be INSIDE a component!  
const [n, setN] = useState('');  
function App() { return <div>{n}</div>; }
```

Correct

```
// Always inside the component function  
function App() {  
  const [n, setN] = useState('');  
  return <div>{n}</div>;  
}
```

Rules of Hooks: useState must always be inside a component. Never outside, in if-statements, or loops.

Re-rendering & Multiple States

How Re-rendering Works

When you call a setter function, here's what React does behind the scenes:



Using Multiple States in One Component

One component can have as many states as it needs. Each one is independent:

```

function UserProfile() {
  // Multiple independent states
  const [name, setName] = useState('Ali');
  const [age, setAge] = useState(25);
  const [isOnline, setIsOnline] = useState(false);
  const [score, setScore] = useState(0);

  return (
    <div>
      <h2>{name} ({age} years old)</h2>
      <p>Status: {isOnline ? 'Online' : 'Offline'}</p>
      <p>Score: {score}</p>
      <button onClick={() => setScore(score + 10)}>
        Add 10 Points!
      </button>
    </div>
  );
}
  
```

The Rules of Hooks (Must Know!)

- ✓ Only call useState at the TOP LEVEL of your component (not inside if, loops, or functions)
- ✓ Only call useState inside FUNCTION components (not class components or regular functions)
- ✓ You can use useState as many times as you want in one component - each is separate
- ✓ useState re-renders ONLY the component where it lives, not the whole app

Challenges & Cheatsheet

Mini Challenges - Test Yourself!

EASY

Build a counter that can go UP (+1) and DOWN (-1). Add a RESET button that sets it back to 0.

MEDIUM

Create a form with name and email inputs. Show a live preview: 'Hello [name], your email is [email]!'

HARD

Build a simple Todo List. User types in an input, clicks Add, item appears in a list with a Delete button.

useState Quick Reference Cheatsheet

```
// ----- useState CHEATSHEET -----

// 1. Import
import { useState } from 'react';

// 2. Basic usage
const [value, setValue] = useState(initialValue);

// 3. Update state (triggers re-render)
setValue(newValue);

// 4. Update based on previous value (safe)
setValue(prev => prev + 1);

// 5. Object state (always spread!)
setUser(prev => ({ ...prev, name: 'Sara' }));

// 6. Array state
setList(prev => [...prev, newItem]); // add
setList(prev => prev.filter(x => x !== item)); // remove
```

You Did It! Keep Building!

useState is the FOUNDATION of React. Master this and every other hook will feel easy!

Keep learning at futurepplx.com | Urdu & Hindi Web Dev Tutorials